

Aspectos Avançados de Banco de Dados

Modelagem NoSQL para Gestão de Academias

Igor Mattos da Motta - 202276006

João Victor Pereira dos Anjos - 202176010

Pedro Detoni Pereira - 202176031

Janeiro de 2026

Conteúdo

1	INTRODUÇÃO	3
1.1	MongoDB - Banco Orientado a Documentos	3
1.2	Neo4j - Banco Orientado a Grafos	3
I	MONGODB - BANCO ORIENTADO A DOCUMENTOS	3
2	MODELAGEM DE DADOS - MONGODB	3
2.1	Entidades (Collections)	3
2.2	Relacionamentos	4
2.3	Índices Implementados	5
3	SCRIPT DE POPULAÇÃO - MONGODB	7
4	CONSULTAS	7
4.1	Consulta 1: Buscar alunos por IMC	7
4.2	Consulta 2: Alunos iniciantes OU com restrições	7
4.3	Consulta 3: Exercícios por grupo muscular	8
4.4	Consulta 4: Média de avaliações por educador	8
4.5	Consulta 5: Treinos mais eficientes (calorias/minuto)	9
4.6	Consulta 6: Educadores com suas avaliações	11
4.7	Consulta 7: Notificações por usuário	11
4.8	Consulta 8: Top 5 execuções mais difíceis	13
4.9	Consulta 9: Alunos com histórico de avaliações	13
4.10	Consulta 10: Estatísticas de mensagens por educador	14
II	NEO4J - BANCO ORIENTADO A GRAFOS	15
5	MODELAGEM DE DADOS (PROPERTY GRAPH)	16
5.1	Componentes do Modelo	16

6	SCRIPT DE CARGA E POPULAÇÃO (SEED SCRIPT)	16
7	CONSULTAS ANALÍTICAS (CYPHER) — QUERIES E RESULTADOS	18
7.1	Consulta 1 — SISTEMA DE RECOMENDAÇÃO	18
7.2	Consulta 2 — ANÁLISE DE CONECTIVIDADE (SHORTEST PATH) . .	19
7.3	Consulta 3 — DETECÇÃO DE CHURN (INATIVIDADE)	19
7.4	Consulta 4 — ALERTA DE OVERTRAINING	19
7.5	Consulta 5 — HEATMAP MUSCULAR (VOLUME POR GRUPO)	20
7.6	Consulta 6 — RANKING DE ASSIDUIDADE	20
7.7	Consulta 7 — ANÁLISE DE IMPACTO (BUSCA REVERSA)	21
7.8	Consulta 8 — VALIDAÇÃO DE INTEGRIDADE	21
7.9	Consulta 9 — CARGA TOTAL PLANEJADA	21
7.10	Consulta 10 — DASHBOARD DO EDUCADOR	22
8	CONCLUSÃO	22
9	REFERÊNCIAS	23

1 INTRODUÇÃO

Este relatório apresenta a implementação de um sistema de gerenciamento para academias utilizando dois paradigmas de bancos de dados NoSQL: MongoDB (orientado a documentos) e Neo4j (orientado a grafos). Cada abordagem oferece vantagens específicas para diferentes aspectos do sistema, permitindo comparação direta entre modelos documentais e de grafos na resolução de problemas reais.

1.1 MongoDB - Banco Orientado a Documentos

O MongoDB é especialmente adequado para armazenar dados hierárquicos e semi-estruturados, oferecendo flexibilidade na modelagem através de documentos JSON/BSON. Permite documentos embarcados, arrays e referências, facilitando a representação de dados complexos sem a necessidade de múltiplas junções.

1.2 Neo4j - Banco Orientado a Grafos

Os bancos de dados em grafo têm se mostrado altamente eficientes para modelar relações complexas e dinâmicas entre entidades. Em ambientes como academias, onde existem vínculos entre educadores, alunos, treinos e exercícios, é vantajoso utilizar um modelo no qual a conectividade é tratada como elemento primário.

Parte I

MONGODB - BANCO ORIENTADO A DOCUMENTOS

Tecnologias Utilizadas:

O projeto foi desenvolvido em **Node.js** utilizando **Mongoose** como ODM (Object Document Mapper) para interação com o MongoDB. O Mongoose fornece validação de schemas, middleware, queries tipadas e abstração sobre o driver nativo do MongoDB.

Repositório: <https://github.com/NashiCodes/Trabalho-AABD>

2 MODELAGEM DE DADOS - MONGODB

A modelagem do sistema de gerenciamento de academias utiliza 8 collections, organizadas para representar as entidades centrais do domínio

A estratégia adotada combina **embedding** (documentos embarcados) e **referencing** (referências por ObjectId) de forma balanceada, considerando os padrões de acesso e os requisitos de consistência.

2.1 Entidades (Collections)

O sistema é composto por 8 collections. Para mais detalhes sobre os schemas, consulte o repositório do projeto.

- **Educadores:** Profissionais responsáveis por orientar alunos e criar treinos. Armazena CREF, especialidades, contato e métricas de avaliação.
- **Alunos:** Usuários do sistema com dados pessoais (email, telefone, nascimento), dados físicos (peso, altura, IMC), objetivos, nível e restrições médicas. Contém histórico de avaliações físicas embarcado.
- **Exercícios:** Catálogo de exercícios com nome único, grupos musculares, tipo, equipamento necessário, nível de dificuldade e calorias estimadas.
- **Treinos:** Planilhas de treino criadas por educadores. Contém lista de exercícios embarcada com configurações específicas (séries, repetições, carga, descanso, ordem).
- **Execuções:** Registro de treinos realizados por alunos, incluindo duração real, calorias queimadas, feedback, percepção de dificuldade e lista de exercícios executados.
- **Avaliações:** Feedback de alunos sobre educadores (nota, comentário, data).
- **Mensagens:** Sistema de mensagens bidirecional entre educadores e alunos (campos polimórficos: remetenteId/Tipo, destinatarioId/Tipo).
- **Notificações:** Sistema de notificações para usuários (campos polimórficos: usuarioId/Tipo), incluindo tipo, título, mensagem e link opcional.

2.2 Relacionamentos

A arquitetura utiliza estratégia híbrida combinando referências (ObjectId) e dados embarcados:

Relacionamentos por Referência (1:N):

- Educadores → Alunos (um educador orienta múltiplos alunos)
- Educadores → Treinos (um educador cria múltiplos treinos)
- Alunos → Execuções (um aluno realiza múltiplas execuções)
- Treinos → Execuções (um treino pode ser executado múltiplas vezes)
- Educadores ← Avaliações (um educador recebe múltiplas avaliações)
- Alunos → Avaliações (um aluno avalia múltiplos educadores)
- Mensagens: Comunicação bidirecional entre Educadores e Alunos
- Notificações: Sistema de alertas para ambos tipos de usuário
- Treinos.exercicios[]: Lista de exercícios com configurações específicas (séries, repetições, carga)
- Execuções.exerciciosRealizados[]: Registro de performance por exercício
- Alunos.historicoAvaliacoes[]: Evolução física do aluno
- Alunos.dadosPessoais, dadosFisicos: Dados agrupados logicamente
- Educadores.contato: Informações de contato

2.3 Índices Implementados

1. Educadores

Índices automáticos (Mongoose):

- `_id` - Índice único criado automaticamente
- `cref` - Índice único (unique: true no schema)
- `contato.email` - Índice único (unique: true no schema)

Índices criados:

- `especialidades`: 1 - Busca por especialidades do educador

2. Alunos

Índices automáticos (Mongoose):

- `_id` - Índice único criado automaticamente
- `dadosPessoais.email` - Índice único (unique: true no schema)

Índices criados:

- `educadorId`: 1 - Busca rápida de alunos por educador
- `nivel`: 1 - Filtragem por nível do aluno
- `objetivos`: 1 - Busca por objetivos do aluno

3. Exercícios

Índices automáticos (Mongoose):

- `_id` - Índice único criado automaticamente
- `nome` - Índice único (unique: true no schema)

Índices criados:

- `grupoMuscular`: 1 - Busca por grupo muscular
- `tipo`: 1 - Filtragem por tipo de exercício
- `nivelDificuldade`: 1 - Busca por dificuldade
- `equipamento`: 1 - Filtrar por equipamento necessário

4. Treinos

Índices automáticos (Mongoose):

- `_id` - Índice único criado automaticamente

Índices criados:

- `criadoPor`: 1 - Busca de treinos por educador criador
- `nivel`: 1 - Filtragem por nível de dificuldade

- objetivo: 1 - Filtragem por objetivo do treino
- tags: 1 - Busca por tags
- {nome: 'text', descricao: 'text'} - Índice de texto completo para pesquisa

5. Avaliações

Índices automáticos (Mongoose):

- _id - Índice único criado automaticamente

Índices criados:

- educadorId: 1 - Busca de avaliações por educador
- alunoId: 1 - Busca de avaliações por aluno
- dataAvaliacao: -1 - Ordenação temporal

6. Execuções

Índices automáticos (Mongoose):

- _id - Índice único criado automaticamente

Índices criados:

- {alunoId: 1, dataHora: -1} - Índice composto para histórico de execuções por aluno
- treinoId: 1 - Busca de execuções por treino
- dataHora: -1 - Ordenação por data
- concluido: 1 - Filtrar treinos concluídos

7. Mensagens

Índices automáticos (Mongoose):

- _id - Índice único criado automaticamente

Índices criados:

- {destinatarioId: 1, lida: 1, dataHora: -1} - Índice composto para caixa de entrada
- {remetenteId: 1, dataHora: -1} - Índice composto para mensagens enviadas

8. Notificações

Índices automáticos (Mongoose):

- _id - Índice único criado automaticamente

Índices criados:

- {usuarioId: 1, lida: 1, dataCriacao: -1} - Índice composto para notificações por usuário
- tipo: 1 - Filtrar por tipo de notificação

3 SCRIPT DE POPULAÇÃO - MONGODB

O script de população é modular, respeitando dependências entre collections:

```
1 const populateDB = async () => {
2   // Fase 1: Entidades base
3   await populateEducadores();
4   await populateAlunos();
5   await populateExercicios();
6
7   // Fase 2: Entidades dependentes
8   await populateTreinos();
9   await populateAvaliacoes();
10  await populateExecucoes();
11  await populateMensagens();
12  await populateNotificacoes();
13 };
```

Listing 1: Orquestrador de População

Dados gerados: 10 educadores, 10 alunos, 10 exercícios, 8 treinos, 10 avaliações, 10 execuções, 10 mensagens, 10 notificações (total:78 documentos).

4 CONSULTAS

4.1 Consulta 1: Buscar alunos por IMC

Objetivo: Demonstrar operadores de comparação \$gte e \$lte.

```
1 Aluno.find({
2   'dadosFisicos.imc': { $gte: 22, $lte: 26 }
3 }).select('nome_dadosFisicos.imc');
```

Listing 2: Query 1 - IMC

Resultado:

Nome	IMC
João Silva	25.6
Maria Oliveira	22.8
Juliana Costa	23.5
Rafael Almeida	24.4
Fernanda Lima	22.3
Lucas Rodrigues	22.6
Camila Ferreira	23.0

Tabela 1: 7 alunos com IMC saudável encontrados

4.2 Consulta 2: Alunos iniciantes OU com restrições

Objetivo: Demonstrar operador lógico \$or.

```

1 Aluno.find({
2   $or: [
3     { nivel: 'Iniciante' },
4     { restricoesMedicas: { $ne: [] } }
5   ]
6 }).select('nome_nivel_restricoesMedicas');

```

Listing 3: Query 2 - Filtro Lógico

Resultado:

Nome	Nível	Restrições
Maria Oliveira	Iniciante	Problemas na coluna
Carlos Eduardo	Avançado	Hipertensão
Ana Paula Santos	Iniciante	Nenhuma
Rafael Almeida	Iniciante	Diabetes tipo 2
Camila Ferreira	Iniciante	Asma

Tabela 2: 5 alunos que precisam atenção especial

4.3 Consulta 3: Exercícios por grupo muscular

Objetivo: Demonstrar operador \$in para arrays.

```

1 Exercicio.find({
2   grupoMuscular: {
3     $in: ['Peitoral', 'Dorsais', 'Ombros']
4   }
5 }).select('nome_grupoMuscular_tipo');

```

Listing 4: Query 3 - Busca em Arrays

Resultado:

Exercício	Grupos Musculares
Remada Curvada	Dorsais, Bíceps, Trapézio
Supino Reto	Peitoral, Tríceps, Ombros
Desenvolvimento Militar	Ombros, Tríceps

Tabela 3: 3 exercícios para parte superior

4.4 Consulta 4: Média de avaliações por educador

Objetivo: Demonstrar agregação com \$group, \$avg e \$lookup.

```

1 Avaliacao.aggregate([
2   {
3     $group: {
4       _id: '$educadorId',

```



```

5     mediaAvaliacoes: { $avg: '$nota' },
6     totalAvaliacoes: { $count: {} }
7   }
8 },
9 {
10    $lookup: {
11      from: 'educadores',
12      localField: '_id',
13      foreignField: '_id',
14      as: 'educador'
15    }
16  },
17  { $unwind: '$educador' },
18  {
19    $project: {
20      nome: '$educador.nome',
21      mediaAvaliacoes: {
22        $round: ['$mediaAvaliacoes', 2]
23      },
24      totalAvaliacoes: 1
25    }
26  },
27  { $sort: { mediaAvaliacoes: -1 } }
28 ]});

```

Listing 5: Query 4 - Aggregation Framework

Resultado:

Educador	Média	Total
Carla Mendes	4.75	4
Fernando Silva	4.67	3
Roberto Almeida	4.33	3

Tabela 4: Ranking de educadores por avaliação

4.5 Consulta 5: Treinos mais eficientes (calorias/minuto)

Objetivo: Demonstrar \$project com expressões matemáticas, \$divide, \$lookup e \$unwind.

```

1 Treino.aggregate([
2   {
3     $match: {
4       duracaoEstimada: { $gt: 0 },
5       caloriasEstimadas: { $gt: 0 }
6     }
7   },
8   {
9     $project: {
10       nome: 1,
11       nivel: 1,

```

```

12     duracaoEstimada: 1,
13     caloriasEstimadas: 1,
14     criadoPor: 1,
15     eficienciaCalórica: {
16         $round: [
17             { $divide: ['$caloriasEstimadas',
18                         '$duracaoEstimada'] },
19             2
20         ]
21     },
22     totalExercicios: { $size: '$exercicios' }
23 }
24 },
25 { $sort: { eficienciaCalórica: -1 } },
26 { $limit: 5 },
27 {
28     $lookup: {
29         from: 'educadores',
30         localField: 'criadoPor',
31         foreignField: '_id',
32         as: 'educador'
33     }
34 },
35 { $unwind: '$educador' },
36 {
37     $project: {
38         nome: 1,
39         nivel: 1,
40         duracaoEstimada: 1,
41         caloriasEstimadas: 1,
42         eficienciaCalórica: 1,
43         totalExercicios: 1,
44         educadorNome: '$educador.nome'
45     }
46 }
47 ]);

```

Listing 6: Query 5 - Cálculos com Expressões

Resultado:

Nota: As colunas duração, calorias e educador foram omitidas para melhor visualização no PDF.

Treino	Nível	Eficiência	Exercícios
Treino HIIT - Emagrecimento	Avançado	15.00 kcal/min	2
Treino Full Body Iniciante	Iniciante	6.22 kcal/min	3
Treino C - Pernas Completo	Avançado	6.11 kcal/min	2
Treino D - Ombros e Abdômen	Intermediário	6.00 kcal/min	2
Treino A - Peito e Tríceps	Iniciante	5.83 kcal/min	2

Tabela 5: Treinos com melhor relação calorias/tempo

4.6 Consulta 6: Educadores com suas avaliações

Objetivo: Usar \$lookup, \$project com \$size, \$avg, e \$addFields com \$max/\$min.

```
1 Educador.aggregate([
2   {
3     $lookup: {
4       from: 'avaliacoes',
5       localField: '_id',
6       foreignField: 'educadorId',
7       as: 'avaliacoes'
8     }
9   },
10  {
11    $project: {
12      nome: 1,
13      especialidades: 1,
14      cref: 1,
15      alunosAtivos: 1,
16      totalAvaliacoes: { $size: '$avaliacoes' },
17      notasRecebidas: '$avaliacoes.nota',
18      avaliacaoCalculada: { $avg: '$avaliacoes.nota' }
19    }
20  },
21  { $match: { totalAvaliacoes: { $gt: 0 } } },
22  {
23    $addFields: {
24      melhorNota: { $max: '$notasRecebidas' },
25      piorNota: { $min: '$notasRecebidas' }
26    }
27  },
28  { $sort: { avaliacaoCalculada: -1 } }
29 ]);
```

Listing 7: Query 6 - Análise de Educadores

Resultado:

Nota: As colunas especialidades e alunos ativos foram omitidas para melhor visualização no PDF.

Educador	CREF	Avaliação	Total	Min/Max
Carla Mendes	234567-G/RJ	4.75	4	4/5
Fernando Silva	345678-G/MG	4.67	3	4/5
Roberto Almeida	123456-G/SP	4.33	3	4/5

Tabela 6: Educadores ordenados por avaliação média

4.7 Consulta 7: Notificações por usuário

Objetivo: Demonstrar \$group com múltiplas agregações, \$lookup duplo e \$ifNull.

```

1  Notificacao.aggregate([
2    {
3      $group: {
4        _id: {
5          usuarioId: '$usuarioId',
6          usuarioTipo: '$usuarioTipo'
7        },
8        totalNotificacoes: { $count: {} },
9        naoLidas: { $sum: { $cond: ['$lida', 0, 1] } },
10       tipos: { $addToSet: '$tipo' },
11       ultimaNotificacao: { $max: '$dataCriacao' }
12     }
13   },
14   { $match: { totalNotificacoes: { $gte: 2 } } },
15   { $sort: { naoLidas: -1, totalNotificacoes: -1 } },
16   { $limit: 5 },
17   {
18     $lookup: {
19       from: 'alunos',
20       localField: '_id.usuarioId',
21       foreignField: '_id',
22       as: 'aluno'
23     }
24   },
25   {
26     $lookup: {
27       from: 'educadores',
28       localField: '_id.usuarioId',
29       foreignField: '_id',
30       as: 'educador'
31     }
32   },
33   {
34     $project: {
35       usuarioTipo: '$_id.usuarioTipo',
36       totalNotificacoes: 1,
37       naoLidas: 1,
38       tipos: 1,
39       nome: {
40         $ifNull: [
41           { $arrayElemAt: ['$aluno.nome', 0] },
42           { $arrayElemAt: ['$educador.nome', 0] }
43         ]
44       }
45     }
46   }
47 ]);

```

Listing 8: Query 7 - Análise de Notificações

Resultado:

Nota: A coluna tipos de notificações foi omitida para melhor visualização no PDF.

Usuário	Tipo	Total	Não lidas
João Silva	Aluno	2	1
Maria Oliveira	Aluno	2	1

Tabela 7: Top usuários com mais notificações

4.8 Consulta 8: Top 5 execuções mais difíceis

Objetivo: Demonstrar \$match, \$sort, \$limit e \$lookup.

```

1 Execucao.aggregate([
2   {
3     $match: {
4       concluido: true,
5       dificuldadePercebida: { $exists: true }
6     }
7   },
8   { $sort: { dificuldadePercebida: -1 } },
9   { $limit: 5 },
10  {
11    $lookup: {
12      from: 'alunos',
13      localField: 'alunoId',
14      foreignField: '_id',
15      as: 'aluno'
16    }
17  },
18  { $unwind: '$aluno' },
19  {
20    $project: {
21      nomeAluno: '$aluno.nome',
22      dificuldadePercebida: 1,
23      duracaoReal: 1,
24      caloriasQueimadas: 1,
25      feedbackAluno: 1
26    }
27  }
28 ]);

```

Listing 9: Query 8 - Ranking de Dificuldade

Resultado:

Nota: A coluna feedback do aluno foi omitida para melhor visualização no PDF.

4.9 Consulta 9: Alunos com histórico de avaliações

Objetivo: Operadores \$exists e \$ne para arrays.

```

1 Aluno.find({
2   historicoAvaliacoes: {
3     $exists: true,
4     $ne: []

```

Aluno	Dificuldade	Duração	Calorias
Pedro Henrique	10/10	32min	465kcal
Ana Paula Santos	9/10	85min	540kcal
Carlos Eduardo	9/10	95min	580kcal
Maria Oliveira	8/10	75min	420kcal
João Silva	7/10	70min	395kcal

Tabela 8: Treinos mais desafiadores reportados

```

5   }
6   }).select('nome_historicoAvaliacoes');

```

Listing 10: Query 9 - Validação de Arrays

Resultado:

Nota: As colunas data da avaliação e percentual de gordura foram omitidas para melhor visualização no PDF.

Aluno	Avaliações	Último Peso
João Silva	1	80kg
Carlos Eduardo	1	95kg
Pedro Henrique	1	85kg
Fernanda Lima	1	59kg

Tabela 9: 4 alunos com histórico físico

4.10 Consulta 10: Estatísticas de mensagens por educador

Objetivo: Pipeline complexo com \$facet.

```

1 Mensagem.aggregate([
2   {
3     $facet: {
4       enviadas: [
5         { $match: { remetenteTipo: 'Educador' } },
6         { $group: {
7           _id: '$remetenteId',
8           totalEnviadas: { $count: {} }
9         }}
10      ],
11      recebidas: [
12        { $match: { destinatarioTipo: 'Educador' } },
13        { $group: {
14          _id: '$destinatarioId',
15          totalRecebidas: { $count: {} }
16        }}
17      ]
18    }
19  },
20  {
21    $project: {
22      mensagens: {

```

```

23     $concatArrays: [ '$enviadas', '$recebidas' ]
24   }
25 }
26 },
27 { $unwind: '$mensagens' },
28 {
29   $group: {
30     _id: '$mensagens._id',
31     totalEnviadas: { $sum: '$mensagens.totalEnviadas' },
32     totalRecebidas: { $sum: '$mensagens.totalRecebidas' }
33   }
34 },
35 {
36   $lookup: {
37     from: 'educadores',
38     localField: '_id',
39     foreignField: '_id',
40     as: 'educador'
41   }
42 },
43 { $unwind: '$educador' },
44 {
45   $project: {
46     nome: '$educador.nome',
47     totalEnviadas: 1,
48     totalRecebidas: 1,
49     total: {
50       $add: [ '$totalEnviadas', '$totalRecebidas' ]
51     }
52   }
53 },
54 { $sort: { total: -1 } }
55 ]));

```

Listing 11: Query 10 - Facet Pipeline

Resultado:

Educador	Enviadas	Recebidas	Total
Carla Mendes	2	2	4
Roberto Almeida	1	2	3
Fernando Silva	1	2	3

Tabela 9: Atividade de comunicação dos educadores

Parte II

NEO4J - BANCO ORIENTADO A GRAFOS

5 MODELAGEM DE DADOS (PROPERTY GRAPH)

A modelagem adotada utiliza o Property Graph Model: nós (labels) e relacionamentos com propriedades. Relacionamentos carregam atributos como séries, repetições e percepção de esforço, facilitando análises que, em modelo relacional, exigiriam tabelas associativas e junções custosas.

5.1 Componentes do Modelo

Labels (Nós):

- Educador
- Aluno
- Treino
- Exercício

Relacionamentos e propriedades relevantes:

- (Educador)-[:ORIENTA]->(Aluno)
- (Educador)-[:CRIOU]->(Treino) // autoria
- (Aluno)-[:SEGUE]->(Treino) // adesão/planilha atual
- (Treino)-[:CONTEM {series, reps}]->(Exercício) // composição da ficha
- (Aluno)-[:REALIZOU {data, percepcao_esforco}]->(Treino) // histórico

Justificativa: armazenar métricas em relacionamentos permite consultas diretas de volume, frequência e percepção de esforço sem junções complexas.

6 SCRIPT DE CARGA E POPULAÇÃO (SEED SCRIPT)

O script abaixo limpa o banco e insere os dados iniciais usados nas consultas. APÊNDICE A (aqui replicado) contém exatamente o script pronto para execução.

(Bloco de comando — formato plain text; aplicar em Neo4j Browser ou cypher-shell)


```

1  /* --- LIMPEZA --- */
2  MATCH (n)
3  DETACH DELETE n;
4
5  /* --- CRIAÇÃO DE EDUCADORES --- */
6  CREATE (pedro:Educador {id: 'e1', nome: 'Pedro', especialidade:
   'Hipertrofia'});
7  CREATE (carla:Educador {id: 'e2', nome: 'Carla', especialidade:
   'Funcional'});
8
9  /* --- CRIAÇÃO DE EXERCÍCIOS --- */
10 CREATE (ex1:Exercicio {nome: 'Supino_Reto', grupo_muscular:
   'Peito'});
11 CREATE (ex2:Exercicio {nome: 'Agachamento_Livre',
   grupo_muscular: 'Pernas'});
12 CREATE (ex3:Exercicio {nome: 'Corrida_na_Esteira',
   grupo_muscular: 'Cardio'});
13 CREATE (ex5:Exercicio {nome: 'Burpee', grupo_muscular: 'Corpo_
   Todo'});
14
15 /* --- CRIAÇÃO DE TREINOS --- */
16 CREATE (t1:Treino {id: 't1', nome: 'Treino_A_-_Força_Bruta',
   foco: 'Força'});
17 CREATE (t2:Treino {id: 't2', nome: 'Treino_Circuito', foco:
   'Emagrecimento'});
18
19 /* --- RELACIONAMENTOS DE AUTORIA --- */
20 CREATE (pedro)-[:CRIOU]->(t1);
21 CREATE (carla)-[:CRIOU]->(t2);
22
23 /* --- COMPOSIÇÃO DOS TREINOS --- */
24 CREATE (t1)-[:CONTEM {series: 5, reps: 5}]->(ex1);
25 CREATE (t1)-[:CONTEM {series: 5, reps: 5}]->(ex2);
26 CREATE (t1)-[:CONTEM {series: 1, reps: 20}]->(ex5);
27 CREATE (t2)-[:CONTEM {series: 3, reps: 15}]->(ex3);
28 CREATE (t2)-[:CONTEM {series: 3, reps: 10}]->(ex5);
29
30 /* --- CRIAÇÃO DE ALUNOS --- */
31 CREATE (a4:Aluno {id: 'a4', nome: 'Carlos', objetivo:
   'Hipertrofia', nivel: 'Intermediário'});
32 CREATE (a7:Aluno {id: 'a7', nome: 'Gustavo', objetivo:
   'Hipertrofia', nivel: 'Intermediário'});
33 CREATE (a8:Aluno {id: 'a8', nome: 'Fernanda', objetivo: 'Saúde',
   nivel: 'Iniciante'});
34 CREATE (a13:Aluno {id: 'a13', nome: 'Marcos', objetivo:
   'Condicionamento', nivel: 'Intermediário'});
35 CREATE (a18:Aluno {id: 'a18', nome: 'Vanessa', objetivo:
   'Funcional', nivel: 'Avançado'});
36
37 /* --- ORIENTAÇÃO --- */
38 CREATE (pedro)-[:ORIENTA]->(a4);

```

```

39 CREATE (pedro)-[:ORIENTA]->(a7);
40 CREATE (pedro)-[:ORIENTA]->(a8);
41 CREATE (carla)-[:ORIENTA]->(a13);
42 CREATE (carla)-[:ORIENTA]->(a18);
43
44 /* --- ADESÃO AOS TREINOS --- */
45 CREATE (a4)-[:SEGUE]->(t1);
46 CREATE (a7)-[:SEGUE]->(t1);
47 CREATE (a7)-[:SEGUE]->(t2);
48 CREATE (a13)-[:SEGUE]->(t2);
49 CREATE (a18)-[:SEGUE]->(t2);
50
51 /* --- HISTÓRICO DE EXECUÇÕES --- */
52 CREATE (a4)-[:REALIZOU {data: date('2023-10-20'),
    percepcao_esforco: 9}]->(t1);
53 CREATE (a4)-[:REALIZOU {data: date('2023-10-22'),
    percepcao_esforco: 9}]->(t1);
54 CREATE (a13)-[:REALIZOU {data: date('2023-10-21'),
    percepcao_esforco: 7}]->(t2);

```

Listing 12: Script de População (Seed)

7 CONSULTAS ANALÍTICAS (CYPHER) — QUÉRIES E RESULTADOS

A seguir estão todas as consultas (Cypher) utilizadas no experimento, seguidas pelos resultados obtidos com os dados do seed acima. Mantive a sintaxe pronta para execução no Neo4j Browser / cypher-shell.

7.1 Consulta 1 — SISTEMA DE RECOMENDAÇÃO

Objetivo: Sugerir treinos baseados em alunos com mesmo nível e objetivo.

```

1 MATCH (carlos:Aluno {nome: 'Carlos'})
2 MATCH (gmeo:Aluno)
3 WHERE gmeo.nivel = carlos.nivel
4 AND gmeo.objetivo = carlos.objetivo
5 AND gmeo.id <> carlos.id
6 MATCH (gmeo)-[:SEGUE]->(t:Treino)
7 WHERE NOT (carlos)-[:SEGUE]->(t)
8 RETURN t.nome AS Sugestao, count(gmeo) AS Popularidade;

```

Listing 13: Consulta 1 - Sistema de Recomendação

Resultado:

Sugestao	Popularidade
Treino Circuito	1

Interpretação: O sistema recomenda “Treino Circuito” para Carlos, pois existe pelo menos um outro aluno (gêmeo de perfil) que segue esse treino.

7.2 Consulta 2 — ANÁLISE DE CONECTIVIDADE (SHORTEST PATH)

Objetivo: Encontrar caminho mínimo entre dois alunos (ex.: Fernanda e Vanessa).

Query:

```
1 MATCH path = shortestPath(  
2 (a1:Aluno {nome: 'Fernanda'})-[*..15]-(a2:Aluno {nome:  
3 'Vanessa'})  
4 )  
5 RETURN path;
```

Listing 14: Consulta 2 - Shortest Path

Resultado: Retorna o caminho mínimo visualizável no Neo4j Browser, por exemplo: Fernanda → (orientado por) Pedro → (ORIENTA) Gustavo → (SEGUE) Treino Circuito → (SEGUE por) Vanessa

(Nota: o grafo permite diversas rotas; o caminho exato é exibido pela função shortestPath)

Interpretação: A rede está conectada entre esses alunos via educadores e treinos, o que possibilita propagação de informações e recomendação via vizinhança.

7.3 Consulta 3 — DETECÇÃO DE CHURN (INATIVIDADE)

Objetivo: Identificar alunos sem histórico de execução (possível churn).

Query:

```
1 MATCH (a:Aluno)  
2 WHERE NOT (a)-[:REALIZOU]->()  
3 RETURN a.nome AS Nome, a.objetivo AS Objetivo;
```

Listing 15: Consulta 3 - Detecção de Churn

Resultado:

Nome	Objetivo
Gustavo	Hipertrofia
Fernanda	Saúde
Vanessa	Funcional

Interpretação: Três alunos não possuem registros de REALIZOU. Observação importante: Gustavo segue treinos (t1 e t2) mas não realizou nenhum — sinal de risco de evasão ou necessidade de engajamento ativo.

7.4 Consulta 4 — ALERTA DE OVERTRAINING

Objetivo: Detectar alunos com percepção de esforço (PSE) ≥ 9 em registros repetidos.

Query:

```
1 MATCH (a:Aluno)-[r:REALIZOU]->(t:Treino)  
2 WHERE r.percepcao_esforco >= 9  
3 WITH a, count(r) AS Qtd
```

```

4 WHERE Qtd >= 2
5 RETURN a.nome AS Aluno, Qtd AS Treinos_Extremos;

```

Listing 16: Consulta 4 - Alerta de Overtraining

Resultado:

Aluno	Treinos_Extremos
Carlos	2

Interpretação: Carlos possui dois registros com PSE $i=9$ — indicador de possível overtraining, requerendo avaliação do educador.

7.5 Consulta 5 — HEATMAP MUSCULAR (VOLUME POR GRUPO)

Objetivo: Calcular volume acumulado por grupo muscular para o aluno Marcos.

Query:

```

1 MATCH (a:Aluno {nome:
   'Marcos'})-[r:REALIZOU]->(t:Treino)-[c:CONTEM]->(ex:Exercicio)
2 WITH ex.grupo_muscular AS Musculo, (c.series * c.reps) AS Vol
3 RETURN Musculo, sum(Vol) AS Volume_Total;

```

Listing 17: Consulta 5 - Heatmap Muscular

Resultado:

Musculo	Volume_Total
Cardio	45
Corpo Todo	30

Cálculo: para cada relação CONTEM, volume = series * reps; somatório por grupo.

7.6 Consulta 6 — RANKING DE ASSIDUIDADE

Objetivo: Ranqueamento de alunos por número total de execuções registradas.

Query:

```

1 MATCH (a:Aluno)-[r:REALIZOU]->(t:Treino)
2 RETURN a.nome AS Aluno, count(r) AS Total
3 ORDER BY Total DESC;

```

Listing 18: Consulta 6 - Ranking de Assiduidade

Resultado:

Aluno	Total
Carlos	2
Marcos	1

Interpretação: Carlos apresenta maior frequência registrada.

7.7 Consulta 7 — ANÁLISE DE IMPACTO (BUSCA REVERSA)

Objetivo: Listar alunos impactados pela indisponibilidade de um exercício (Ex.: “Corrida na Esteira”).

Query:

```
1 MATCH (ex:Exercicio {nome: 'Corrida na Esteira'}) <-[:CONTEM]-(t:Treino)<-[:SEGUE]-(a:Aluno)
2 RETURN t.nome AS Treino, collect(a.nome) AS Alunos_Afetados;
```

Listing 19: Consulta 7 - Análise de Impacto

Resultado:

Treino	Alunos_Afetados
Treino Circuito	[Gustavo, Marcos, Vanessa]

Interpretação: Manutenção na esteira impacta diretamente esses alunos; ação operacional necessária (comunicação, substituição de exercício).

7.8 Consulta 8 — VALIDAÇÃO DE INTEGRIDADE

Objetivo: Encontrar alunos que têm educador vinculado, mas não seguem nenhum treino.

Query:

```
1 MATCH (a:Aluno)
2 WHERE NOT (a)-[:SEGUE]->(t:Treino)
3 MATCH (p:Educador)-[:ORIENTA]->(a)
4 RETURN a.nome AS Aluno, p.nome AS Professor;
```

Listing 20: Consulta 8 - Validação de Integridade

Resultado:

Aluno	Professor
Fernanda	Pedro

Interpretação: Fernanda está orientada por Pedro, mas não possui ficha associada; requer verificação administrativa/operacional.

7.9 Consulta 9 — CARGA TOTAL PLANEJADA

Objetivo: Somar repetições teóricas por treino (soma de series * reps).

Query:

```
1 MATCH (t:Treino)-[:CONTEM]->(ex:Exercicio)
2 RETURN t.nome AS Treino, sum(r.series * r.reps) AS Reps_Totais;
```

Listing 21: Consulta 9 - Carga Total Planejada

Treino	Reps_Totais
Treino Circuito	75
Treino A - Força Bruta	70

Resultado:

Cálculo exemplo (Treino Circuito): Corrida na Esteira: 3 séries x 15 reps = 45
 Burpee: 3 séries x 10 reps = 30
Total = 45 + 30 = 75

7.10 Consulta 10 — DASHBOARD DO EDUCADOR

Objetivo: Visão consolidada da rede do educador (alunos orientados e treinos criados).

Query:

```

1 MATCH (p:Educador {nome: 'Pedro'})
2 OPTIONAL MATCH (p)-[:ORIENTA]->(a:Aluno)
3 OPTIONAL MATCH (p)-[:CRIOU]->(t:Treino)
4 RETURN p.nome AS Educador,
5 collect(DISTINCT a.nome) AS Alunos,
6 collect(DISTINCT t.nome) AS Treinos;
```

Listing 22: Consulta 10 - Dashboard do Educador

Resultado (retorno):

Educador	Alunos	Treinos
Pedro	[Carlos, Gustavo, Fernanda]	[Treino A - Força Bruta]

Interpretação: Visão operacional para o professor Pedro gerenciar alunos e treinos.

8 CONCLUSÃO

Este trabalho implementou um sistema de gerenciamento de academias utilizando dois paradigmas NoSQL complementares: MongoDB (orientado a documentos) e Neo4j (orientado a grafos).

O **MongoDB**, com sua modelagem híbrida de embedding e referencin, demonstrou flexibilidade para representar dados hierárquicos e semi-estruturados. O uso do Mongo-ose ODM proporcionou validação de schemas, indexação automática e queries tipadas. As 10 consultas analíticas desenvolvidas exploraram recursos avançados do Aggregation Framework (\$lookup, \$group, \$facet, \$project), demonstrando capacidade para análises complexas como eficiência calórica, ranking de educadores e estatísticas de comunicação.

O **Neo4j**, através do modelo Property Graph, permitiu modelar relacionamentos de forma natural, resultando em consultas expressivas para recomendação, análise de churn, detecção de overtraining e impacto operacional. A linguagem Cypher simplificou consultas que, em modelos relacionais, exigiriam múltiplas junções.

Ambas as abordagens apresentam trade-offs: MongoDB favorece agregações e queries hierárquicas, enquanto Neo4j excele em análises de conectividade e traversal de grafos.

A escolha entre eles deve considerar requisitos específicos de consulta, volume de dados e complexidade relacional.

9 REFERÊNCIAS

MONGODB. **MongoDB Documentation**. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 23 jan. 2026.

MONGODB. **MongoDB Manual: Aggregation Pipeline**. Disponível em: <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>. Acesso em: 23 jan. 2026.

AUTOMATTIC. **Mongoose ODM Documentation**. Disponível em: <https://mongoosejs.com/docs/>. Acesso em: 23 jan. 2026.

CHODOROW, K.; DIROLF, M. **MongoDB: The Definitive Guide**. 3rd ed. O'Reilly Media, 2019.

NEO4J. **Neo4j Graph Database**. Disponível em: <https://neo4j.com>. Acesso em: 22 jan. 2026.

NEO4J. **Cypher Query Language Documentation**. Disponível em: <https://neo4j.com/docs/cypher-manual/>. Acesso em: 22 jan. 2026.